



FT5101 - FinTech Research Immersion

National University of Singapore
Asian Institute of Digital Finance

KOL2Trade: A Reinforcement Learning Framework for KOL-Informed Trading

Yuncong Liu - ID: A0332507L

November 2025

Contents

1 Literature Review	3
2 Methodology	4
2.1 Motivation and Problem Formulation	4
2.2 Text-Driven Baseline Policy and Replay Buffer	4
2.3 Dual-Branch Action Policy	5
2.4 Offline RL Training with BC and IQL	5
3 Experiments	6
3.1 Dataset	6
3.2 Training Setup	6
3.3 Evaluation Protocol	6
3.4 Results	6
4 Conclusion	7
5 Future Work	7
A KOL Structural Signal Characteristics	8
B Experiment Setup	9
C Model Architecture	11
D Full Experimental Results	12
E Action-Level Comparison Between Baseline and Trained Policy	15

Abstract. KOL (Key Opinion Leader) content provides clear guidance on what to buy, but converting such free-form text into full trading trajectories remains underexplored. We present **KOL2Trade**, an end-to-end framework that maps KOL textual signals to structured multi-asset portfolio actions. A sentiment–confidence model produces an interpretable baseline that captures KOL-driven entry and sizing decisions under long–short and exposure constraints.

Real-world KOL text rarely specifies how long to hold or when to exit. To complete the strategy, we introduce an offline RL agent that learns position evolution beyond the initial entry. A dual-branch policy preserves KOL intent when signals appear while learning decay or exit behavior during silent periods, trained with BC + IQL over a replay buffer incorporating text, ticker, position, and silence features.

Experiments show that KOL2Trade faithfully reproduces KOL-style entries while learning effective holding and exit dynamics that the text omits, yielding an interpretable and market-ready KOL-aligned trading agent.

1 Literature Review

Text-driven trading. Textual information—including news, social media, and analyst commentary—carries predictive signals for asset returns. Early work links aggregate sentiment to market movements (**Twitter-sentiment** [1]), while later studies use event extraction to identify firm-level catalysts (**Event-driven models** [2]). Modern language models enhance financial text understanding (**BERT-finance** [3]), and retail platforms such as StockTwits and Reddit show that user-generated opinions influence volatility and attention (**Retail-attention** [4], **StockTwits-sentiment** [5]). Most of this literature treats text as a predictive feature rather than explicit trading intent—the perspective we adopt in KOL2Trade.

KOL and influencer-driven signals. Influencer-generated content has increasing impact on retail trading flows (**Influencer-effects** [6]). Compared with aggregate sentiment, KOL commentary provides more direct guidance on asset selection and direction, aligning with emerging “text-to-action” paradigms in which language is interpreted as actionable decisions rather than sentiment proxies (**Text-to-Action** [7]). Our work builds on this shift by mapping KOL text into per-asset portfolio intentions.

Reinforcement learning for portfolio management. RL has been widely explored for trading, including deep Q-learning, actor–critic methods, and direct policy optimisation (**DeepRL-trader** [8], **Actor-critic portfolio** [9], **Direct-RL** [10]). Because online exploration is risky in financial markets, recent work focuses on offline RL, where **CQL** [11] and **IQL** [12] mitigate distribution shift and instability in noisy datasets.

Gap and our direction. Despite advances in NLP-driven trading and RL-based portfolio optimisation, these research threads remain largely separate: NLP models use text mainly for prediction, while RL methods typically ignore language as a grounding signal for trading actions. Moreover, KOL-style content exhibits strong entry–exit asymmetry (Appendix 1): positive mentions dominate, 3–11% of tickers appear only once, silence periods often last months, and reversals are rare. Thus KOL text provides clear **entries** but little guidance for **holding** or **exit** decisions.

KOL2Trade addresses these gaps by extracting KOL intent, mapping it into interpretable portfolio actions, and using offline RL to learn the missing holding and exit dynamics—bridging natural-language investment commentary with sequential decision-making in a way not covered by prior literature.

2 Methodology

2.1 Motivation and Problem Formulation

KOL commentary provides clear **entry** guidance but almost no information on **holding** or **exit**. Over 40% of tickers are mentioned only once, silence periods often exceed 100 days, and explicit reversals are rare. Let $x_t^{(i)}$ denote KOL text for ticker i on day t : when $x_t^{(i)}$ exists, an entry signal is given; when $x_{t+k}^{(i)} = \emptyset$, the text provides no further instruction.

Our goal is therefore twofold: (i) encode KOL intent into structured, interpretable portfolio actions when signals appear, (ii) learn how positions should evolve during silent periods. We address this by combining a transparent text-driven baseline with an offline RL agent that learns holding and exit behaviour from logged returns.

2.2 Text-Driven Baseline Policy and Replay Buffer

Each mention produces a sentiment–confidence score

$$\text{raw}_t^{(i)} = \tanh\left(2 \text{sentiment}_t^{(i)} \cdot \text{confidence}_t^{(i)}\right),$$

which is normalized into long–short weights

$$b_t^{(i)} = \frac{\text{raw}_t^{(i)}}{\sum_j |\text{raw}_t^{(j)}|} \quad \text{s.t.} \quad |b_t^{(i)}| \leq w_{\max}.$$

Unmentioned tickers inherit the previous position $b_{t-1}^{(i)}$, reflecting the absence of exit cues.

To support silent-period learning, the replay buffer stores

$$s_t = \left[\text{BERT}(x_t), \text{ticker_emb}(i), \text{sentiment}_t, \text{confidence}_t, \text{last_pos}_t^{(i)}, \text{silence_days}_t^{(i)}, h_t^{(i)} \right],$$

where $h_t^{(i)} = 1$ indicates a mention. This captures both explicit intent (text/sentiment) and implicit temporal structure (silence).

2.3 Dual-Branch Action Policy

To preserve KOL intent while enabling learned exit behaviour, we use a dual-branch actor.

****Signal branch:**** when $h_t^{(i)} = 1$,

$$a_{\text{sig},t}^{(i)} = b_t^{(i)} \left(1 + \alpha \tanh(\delta_{\text{sig},t}^{(i)}) \right),$$

which scales but never reverses baseline direction.

****Silent branch:**** when $h_t^{(i)} = 0$,

$$a_{\text{nosig},t}^{(i)} = \text{last_pos}_t^{(i)} \sigma(\beta \delta_{\text{decay},t}^{(i)}),$$

allowing the model to maintain, gradually reduce, or exit stale positions according to reward patterns and silence duration.

The overall action is

$$a_t^{(i)} = h_t^{(i)} a_{\text{sig},t}^{(i)} + (1 - h_t^{(i)}) a_{\text{nosig},t}^{(i)},$$

and is mapped into a valid portfolio via normalized weights $w_t^{(i)} = a_t^{(i)} / \sum_j |a_t^{(j)}|$.

2.4 Offline RL Training with BC and IQL

The signal branch receives a light BC prior,

$$\mathcal{L}_{\text{BC}} = \|a_{\text{sig},t}^{(i)} - b_t^{(i)}\|^2,$$

ensuring fidelity to KOL intent. The silent branch receives no BC, allowing unbiased learning of decay/exit decisions.

IQL fits value targets via expectile regression,

$$V(s_t) = \arg \min_v \mathbb{E}[\rho_\tau(Q(s_t, a_t) - v)],$$

with critic update

$$Q(s_t, a_t) = r_t + \gamma V(s_{t+1}),$$

and actor optimization through advantage-weighted regression:

$$\mathcal{L}_{\text{actor}} = -\exp(\beta A(s_t, a_t)) + \lambda \|a_{\text{sig},t}^{(i)} - b_t^{(i)}\|^2.$$

The fidelity term applies only when $h_t^{(i)} = 1$, ensuring faithful entry behaviour while granting full flexibility during silent periods.

Together, these components form a complete lifecycle policy: text governs entries, and offline RL learns data-driven holding and exit dynamics that the text omits.

3 Experiments

3.1 Dataset

We evaluate KOL2Trade on three major KOLs: `Everything_Money`, `Invest_with_Henry`, and `MarketBeat`. All 2024 videos are converted into day-level signals and aligned with daily OHLCV prices. Train/validation/test splits follow chronological order to ensure strict causality. Replay buffers store text/ticker embeddings, sentiment and confidence scores, last positions, silence features, baseline actions, and realized rewards. Summary statistics of KOL signal asymmetry appear in Appendix A.

3.2 Training Setup

Policies are trained using the BC+IQL residual framework (Section 2). The signal branch receives a weak BC prior, while the silent-period branch is trained purely from advantage-weighted regression. Unless stated otherwise, we use: 200k IQL steps, batch size 256, expectile $\tau = 0.7$, temperature $\beta = 3$, residual/decay scales in $[0.2, 1.0]$, and a 20% per-asset exposure cap. Implementation details and commands are provided in Appendix B.

3.3 Evaluation Protocol

On validation/test splits, we replay the learned policy day by day: restore the previous portfolio, apply residual-modified weights, and accumulate NAV. We compare four strategies: **Baseline1** (strict KOL replication), **Trained** (residual policy), **Baseline2** (free model without KOL constraints), and **SPY**. We report cumulative return, Sharpe ratio, max drawdown, and fidelity deviation. Full metrics and equity curves are in Appendix D.

3.4 Results

Overall Performance. Across all KOLs, the trained policy matches or exceeds the strict KOL baseline while maintaining substantially better stability than the unconstrained free model. Sharpe ratios improve in most cases, and drawdowns remain well-controlled.

Silent-Period Learning. The largest gains arise during days without KOL guidance. The RL agent learns to delay liquidation, trim selectively, and avoid unnecessary churn during long silence periods—behaviors that improve NAV trajectories and reduce noisy reversals. Action-level examples are shown in Appendix E.

Case Study: `Invest_with_Henry`. Decision cards (Appendix E) show that the trained agent achieves better-timed reductions (e.g., TSLA/NVDA on 11-07 / 11-09) and avoids over-selling strong names (e.g., MSFT on 11-12). Across 10 representative days, 7 yield positive NAV improvement, closely matching the overall gains in Appendix D.

Interpretation. The results reveal a clear division of labor: KOL text specifies **what** to buy or sell, while the RL agent learns **how long** to hold and **when** to exit. This produces an interpretable, complete trading workflow that respects each KOL’s style while correcting for incomplete exit guidance.

4 Conclusion

This work introduces **KOL2Trade**, a framework that transforms free-form KOL commentary into actionable multi-asset trading decisions. By combining a transparent text-driven baseline with an offline RL agent, the system captures the strengths of KOL guidance—clear entry intent—while learning the missing lifecycle components (effective holding, decay, and exit) that the text does not specify. Experiments across three high-volume KOLs show that the learned policy improves risk-adjusted performance and delivers more stable position management than both strict replication and unconstrained models. The dual-branch architecture provides interpretability aligned with real-world trading needs and preserves the identifiable style of each KOL.

5 Future Work

Several directions remain for extending KOL2Trade:

- **Scaling to larger and more diverse datasets.** Incorporating a broader set of KOLs, multi-platform content (e.g., X/Twitter, TikTok, livestream transcripts), and longer historical windows could reveal richer behavioural patterns and improve generalization.
- **Advanced training objectives.** Future work may integrate uncertainty-aware RL, sequence modeling (e.g., decision transformers or trajectory models), and stronger language–action alignment objectives to better capture nuanced guidance and long-range dependencies in commentary.
- **Live deployment and closed-loop evaluation.** A natural next step is real-time or paper-trading deployment to examine how the agent performs under market microstructure noise, execution frictions, and rapidly changing KOL behaviour.

KOL2Trade provides a foundation for text-to-action trading agents, and we expect that future developments in language modeling and offline RL will further enhance the practicality and interpretability of such systems.

Appendix A KOL Structural Signal Characteristics

Table 1: Structural signal characteristics of the three KOL datasets.

KOL	Pos%	Neg%	Singleton Rate	Median Silence (days)	90% Silence (days)	Flip Rate
Everything Money	58.5	19.9	4.6%	54.5	187.5	42.1%
Invest with Henry	66.7	9.4	3.5%	22.5	87.0	44.4%
MarketBeat	72.2	9.3	10.7%	104.0	259.0	18.0%

Appendix B Experiment Setup

This appendix provides detailed information about the dataset, preprocessing pipeline, reward construction, and train/validation/test splits for the three KOLs used in evaluation.

B.1 Dataset and Time Range

All experiments use content published between **2024-01-01** and **2024-12-31**. Each KOL is treated as an independent dataset and split chronologically by publication date.

Table 2: Train/validation/test splits for each KOL (by published date).

KOL	Train	Validation	Test
Everything_Money	552 videos	155 videos	187 videos
Invest_with_Henry	302 videos	193 videos	163 videos
MarketBeat	341 videos	62 videos	90 videos

B.2 Reward Construction

Daily reward is computed as the next-day realized portfolio return:

$$r_t = \sum_i w_t^{(i)} \cdot \left(\frac{p_{t+1}^{(i)}}{p_t^{(i)}} - 1 \right)$$

where $w_t^{(i)}$ denotes the portfolio weight produced by the agent.

B.3 Replay Buffer Contents

For each day t and ticker i , the replay buffer stores:

- BERT text embedding of the KOL content
- ticker embedding
- sentiment and confidence scores
- last position $b_{t-1}^{(i)}$
- silence duration $\text{silence_days}_t^{(i)}$
- binary signal flag $h_t^{(i)}$

- baseline action $b_t^{(i)}$
- reward r_t

Full schema is provided in the project code and dataset documentation.

Appendix C Model Architecture

This appendix describes the full architecture and hyperparameters used in training the KOL2Trade agent.

C.1 Policy Architecture

- **Actor:** 3-layer MLP with dual heads:

$$\delta_{\text{sig}}, \delta_{\text{decay}}$$

- **Critic / Value Network:** 3-layer MLP
- **Action parameterization:**

$$a_{\text{sig}} = b_t^{(i)} (1 + \alpha \tanh(\delta_{\text{sig}}))$$

$$a_{\text{nosig}} = b_{t-1}^{(i)} \sigma(\beta \delta_{\text{decay}})$$

C.2 Offline RL Training (IQL)

- Expectile $\tau = 0.7$
- Temperature $\beta = 3$
- Learning rate 3×10^{-4}
- Batch size 256
- IQL steps: 200k
- BC warmup epochs: 1 (signal branch only)

C.3 Hyperparameter Values

- `residual_scale` $\in \{0.2, 0.3\}$
- `decay_scale` $\in \{0.5, 1.0\}$
- `fidelity_lambda` $\in \{0, 0.1, 0.3\}$ (default: 0)
- `max_weight` = 0.2

Appendix D Full Experimental Results

This appendix presents full quantitative results for the three KOLs, including cumulative returns, Sharpe ratios, maximum drawdowns, and fidelity deviation. Equity curves are provided for all four strategies: KOL baseline, trained residual policy, free policy, and SPY.

D.1 Complete Metrics

Table 3: Evaluation metrics for MarketBeat test set.

Strategy	Return	Sharpe	Max DD	Fidelity Dev
Baseline1_KOL	-0.0048	0.391	-0.0211	0.000
Trained Residual	-0.0042	0.366	-0.0221	0.152
Baseline2_Free	0.0050	0.326	-0.0389	0.786
SPY	0.0185	0.875	-0.0173	N/A

Table 4: Evaluation metrics for Invest_with_Henry test set.

Strategy	Return	Sharpe	Max DD	Fidelity Dev
Baseline1_KOL	0.0704	3.017	-0.0154	0.000
Trained Residual	0.1005	2.921	-0.0206	0.244
Baseline2_Free	0.0529	1.694	-0.0247	0.811
SPY	0.0112	0.445	-0.0195	N/A

Table 5: Evaluation metrics for Everything_Money test set.

Strategy	Return	Sharpe	Max DD	Fidelity Dev
Baseline1_KOL	0.1071	2.973	-0.0228	0.000
Trained Residual	0.1249	3.211	-0.0264	0.190
Baseline2_Free	0.0424	1.263	-0.0362	0.808
SPY	0.0178	0.899	-0.0188	N/A

D.2 Equity Curves

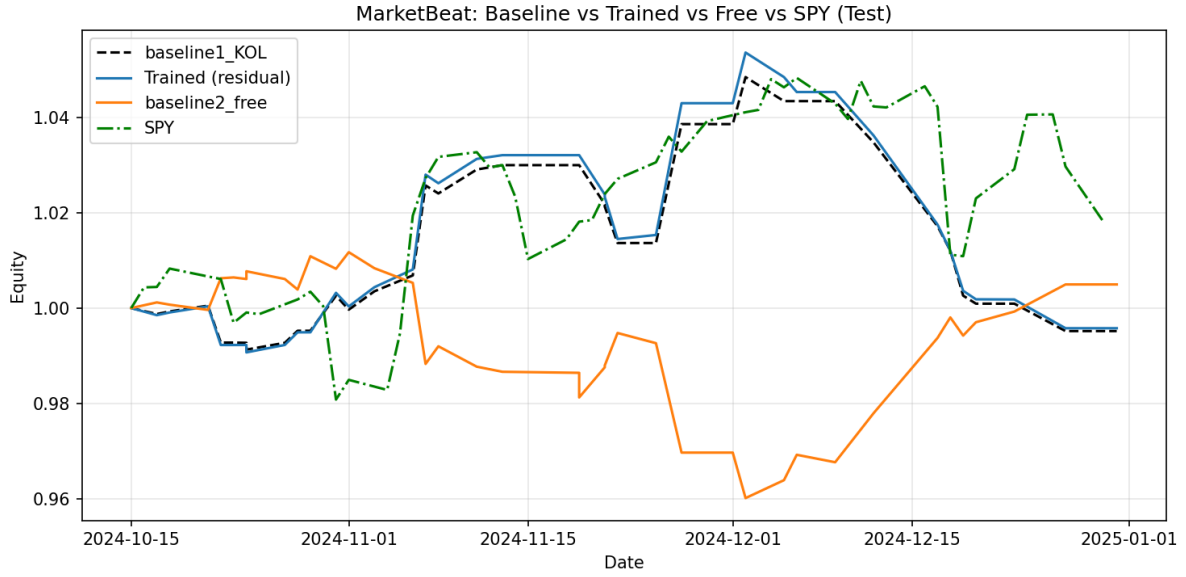


Figure 1: MarketBeat: Equity curves of baseline, trained residual, free policy, and SPY.

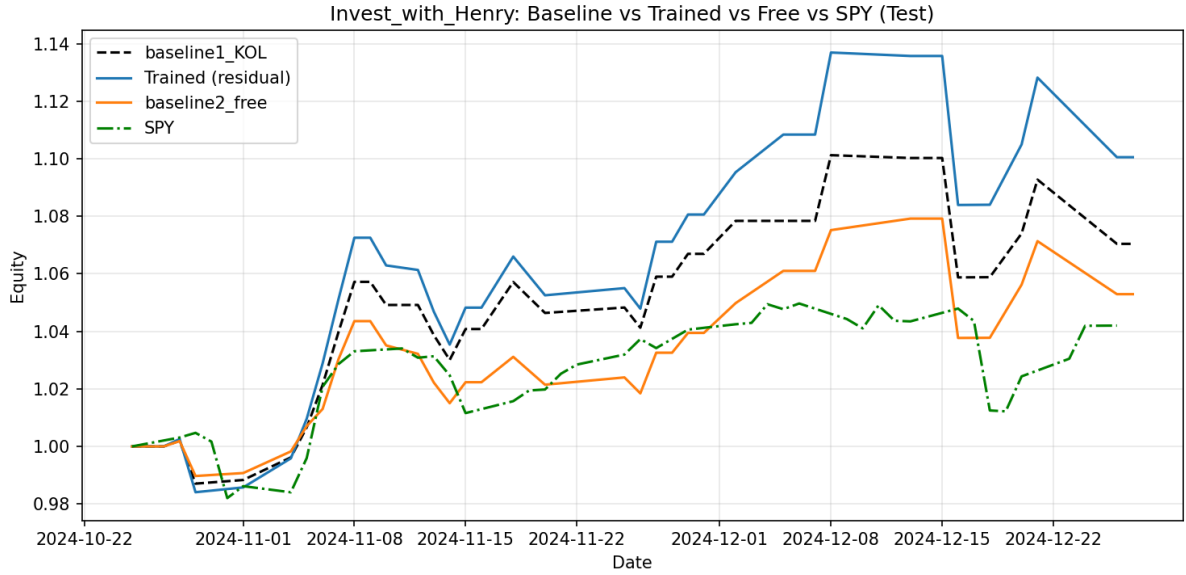


Figure 2: Invest_with_Henry: Equity curves of all strategies.

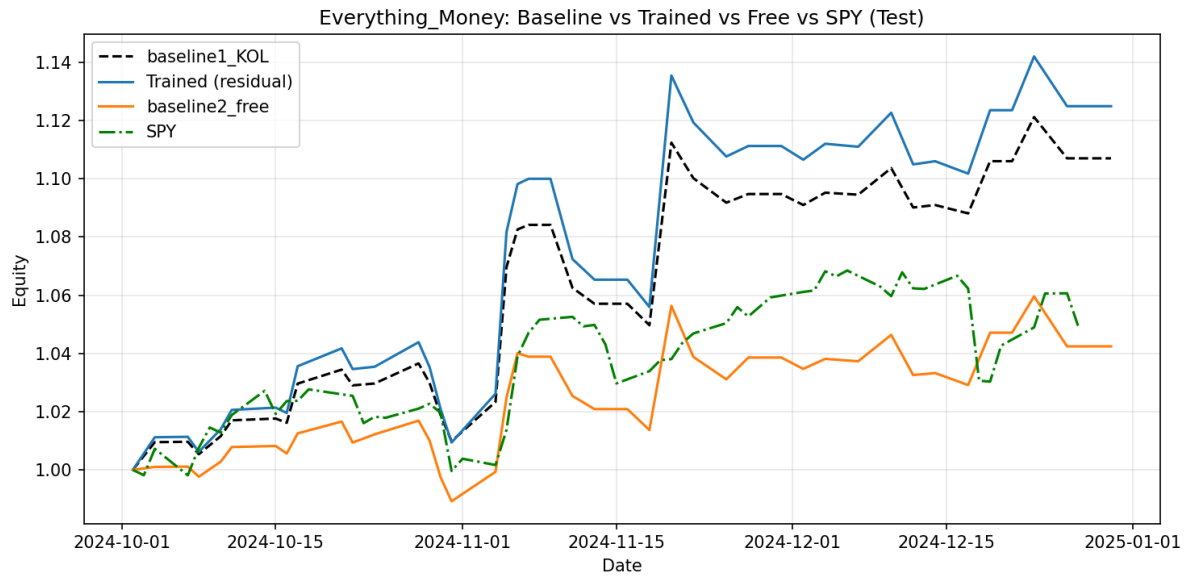


Figure 3: Everything_Money: Equity curves of all strategies.

Appendix E Action-Level Comparison Between Baseline and Trained Policy

This appendix compares baseline actions with the trained residual policy on ten key decision days for `Invest_with_Henry`. Each card displays two mirrored panels to highlight exactly how the trained policy modifies KOL-intended trades.

Notation. NAV refers to cumulative portfolio equity, normalized to 1 on the first test day. An entry such as “1.0398→1.0442” indicates that the trained policy ends the day with higher equity (1.0442) compared to the baseline (1.0398).

2024-11-07	NAV: 1.0398 → 1.0442
Baseline Actions AAPL close; KR open; PAYC increase; TSLA/NVDA large decreases.	Trained Actions AAPL partial reduction; KR open; PAYC increase; deeper TSLA/NVDA trimming.
Effect: Higher NAV; better focus. NAV Impact: The trained policy increased daily equity by +0.44% (1.0398→1.0442).	

2024-11-08	NAV: 1.0602 → 1.0654
Baseline Actions NVDA increase; AAPL decrease; TSLA further increase.	Trained Actions NVDA increase; AAPL hold; TSLA increase.
Effect: Higher NAV; less aggressive AAPL selling. NAV Impact: The trained policy achieved a +0.52% improvement over the baseline.	

2024-11-09	NAV: 1.0602 → 1.0654
Baseline Actions Multiple opens (WMT/KHC/ABNB/MSFT/XOM/...); AAPL/TSLA/NVDA decrease; PYPL decrease.	Trained Actions PYPL hold; more conservative TSLA/NVDA reductions.

Effect: More conservative reductions; retains small old positions.

NAV Impact: The trained policy produced a +0.52% higher NAV than the baseline.

2024-11-12

NAV: 1.0508 → 1.0543

Baseline Actions

AAPL/MSFT/NVDA/TSLA all close.

Trained Actions

Partial exits for NVDA/TSLA; MSFT hold; AAPL reopened small.

Effect: Avoids over-liquidation.

NAV Impact: The trained policy improved equity by +0.35% relative to baseline.

2024-11-13

NAV: 1.0386 → 1.0399

Baseline Actions

MSFT/NVDA/TSLA open evenly.

Trained Actions

Sharper concentration on stronger signals.

Effect: Better allocation.

NAV Impact: The trained policy yielded a +0.13% NAV advantage.

2024-11-15

NAV: 1.0410 → 1.0413

Baseline Actions

PYPL/SBUX increase; AAPL open; NVDA/TSLA decrease; GS hold.

Trained Actions

PYPL smaller open; TSLA larger decrease; AAPL small.

Effect: More cautious adds; stronger focus.

NAV Impact: The trained policy provided a marginal +0.03% benefit.

2024-11-18		NAV: 1.0601 → 1.0589	
Baseline Actions GS close; TSLA/NVDA increase.		Trained Actions GS partial exit; TSLA/NVDA more concentrated.	
Effect: Concentration increases. NAV Impact: The trained policy underperformed by −0.12%.			
2024-11-20		NAV: 1.0476 → 1.0455	
Baseline Actions NVDA/TSLA decrease; GS open.		Trained Actions NVDA/TSLA similar decrease; GS increased aggressively.	
Effect: Aggressive shift. NAV Impact: The trained policy resulted in a −0.21% lower NAV.			
2024-11-26		NAV: 1.0416 → 1.0409	
Baseline Actions TSLA decrease; NVDA increase; AAPL increase; GS close.		Trained Actions TSLA decrease; NVDA decrease; AAPL increase; GS partial exit.	
Effect: More cautious adjustment. NAV Impact: The trained policy was −0.07% below the baseline.			
2024-11-27/28		NAV: 1.0623 → 1.0640	
Baseline Actions 11/27: NVDA decrease; TSLA increase; GS hold. 11/28: NVDA decrease; AAPL hold; TGT open.		Trained Actions 11/27: NVDA increase; TSLA increase; GS close. 11/28: AAPL increase; TGT open.	
Effect: Stronger signal following. NAV Impact: The trained policy improved NAV by +0.17%.			

References

- [1] Johan Bollen, Huina Mao, and Xiaojun Zeng. Twitter mood predicts the stock market. [Journal of Computational Science](#), 2011. pages 3
- [2] Xiao Ding, Yue Zhang, Ting Liu, and Junwei Duan. Deep learning for event-driven stock prediction. In [IJCAI](#), 2015. pages 3
- [3] Qin Chen et al. Enhancing stock movement prediction with adversarial training. [ACL](#), 2019. pages 3
- [4] Brad Barber, Xing Huang, Terrance Odean, and Christopher Schwarz. Retail investor attention and the gamestop short squeeze. [NBER Working Paper](#), 2021. pages 3
- [5] Jun Hao Soh et al. Predictive power of stocktwits sentiment for equity returns. [Finance Research Letters](#), 2022. pages 3
- [6] Giovanna Anzellotti et al. The role of social influencers in online investment communities. [SSRN Working Paper](#), 2022. pages 3
- [7] Yiming Yang et al. Text-to-action: Mapping natural language to executable trading decisions. In [NeurIPS Workshop](#), 2023. pages 3
- [8] Zhengyao Jiang, Dixing Xu, and Jinjun Liang. Deep reinforcement learning for unsupervised portfolio management. In [ICDM](#), 2017. pages 3
- [9] Thomas Fischer and Christopher Krauss. Reinforcement learning for financial portfolio management. [Expert Systems with Applications](#), 2018. pages 3
- [10] John Moody and Matthew Saffell. Learning to trade via direct reinforcement. [IEEE Transactions on Neural Networks](#), 2001. pages 3
- [11] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. [NeurIPS](#), 2020. pages 3
- [12] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. In [ICLR](#), 2022. pages 3